

BEACON LABS

CYBERSECURITY EDUCATION LABS

Android Adaptive Activity Hijacking Attack Lab

Module: Mobile Security



BOISE STATE UNIVERSITY

COLLEGE OF ENGINEERING

Department of Computer Science

Table of Contents

Overview	3
Setup	4
Task Set 1: Getting Device Information & Sending it to a Remote Server	6
Task 1.1: Getting Device Information	6
Task 1.2: Creating the Remote Server	7
Task 1.3: Connecting and Communicating w/ the Remote Server	8
Task Set 2: Recreating App w/ Appium and Creating Patchfile	10
Task 2.1: Accessing Device via ADB Shell	10
Task 2.2: Connecting to Device Activity via Appium	12
Task 2.3 Generating Fake UI & Patchfile	15
Task Set 3: Game Theory	16
Submission	20
Summary	21
Trouble Shooting	21
Refactoring the Project Name	21
References	21

Overview

Activity hijacking attacks were once one of the most powerful attacks against Android devices, but thanks to the presence of new effective defense mechanisms, they typically no longer pose security threats in more recent versions.

In this exercise, we explore a new state-of-the-art automated and adaptive hijacking attack that utilizes a spectrum of customized attacks. Specifically, we explore the use of hot patch techniques to identify vulnerable devices and update attack payload without the need for re-installation/re-distribution.

Furthermore, we'll utilize this service to receive a list of installed applications from the device via a remote server in order to determine a target application and recreate it utilizing an automated UI-generator tool known as Appium.

Objectives

Upon completing this lab you should know how to connect an application to a remote server and returning device information in the background, utilize a hot patch framework to update application without need for re-installation, and generate automated UI tests with Appium.

Reading

This lab is based on a paper by P. Sun, S. Chen, L. Fan, P. Gas, F. Song, and M. Yang,^[1] and was published in the Frontiers of Computer Science. Reading the additional paper is not necessary but can help if you would like to learn more about this attack.

[1] P. Sun, S. Chen, L. Fan, P. Gas, F. Song, and M. Yang, "VenomAttack: Automated and Adaptive Activity Hijacking in Android", 2023.

Setup

A provided zipped folder contains all necessary files in order to run and complete this lab. It is recommended that you have 8 GB RAM or more, x86_64 CPU architecture; 2nd generation Intel Core or newer, or AMD CPU with support for a Windows Hypervisor / ARM-based chips, or 2nd generation Intel Core or newer with support for Hypervisor.Framework, and 9 GB of available disk space minimum or more.

Upon opening the provided lab folder in Android Studio, open a terminal into the topmost VenomAttack directory and run the command:

```
$ git init
```

The above command will initialize a new git repository which will be used in Action 2.3.2. You do not need to create a GitHub repository or commit your progress as you complete the lab.

First create your emulated device. On the right hand side of your screen, click the 'Device Manager' tab, and select 'Create device'. Scroll down and select the 'Pixel 5' option. When choosing your system image, select the second tab that corresponds to your CPU architecture (x86 / ARM). Scroll down and select API 28 for Android 9 Pie. Make sure your application binary interface corresponds to your matching architecture. Lastly, make sure beneath the target column that your system image does not utilize Google API's or services.

Now that you have created your device, run the device to confirm that it does so successfully. If any issues occur, refer to the appendix below or consult with Google. You will notice your device will be in a factory-stock state with the default applications installed. Navigate to your MainActivity class beneath the java application folder, right click it, and select "Run 'MainActivity'". This will cause the emulator to launch your device and install the application.

Next, install the latest version of [Appium Server](#) on your respective device. Navigate to the link and locate the 'Releases' section on the right-hand side of the page. Below you will find a list of images beneath the 'Assets'

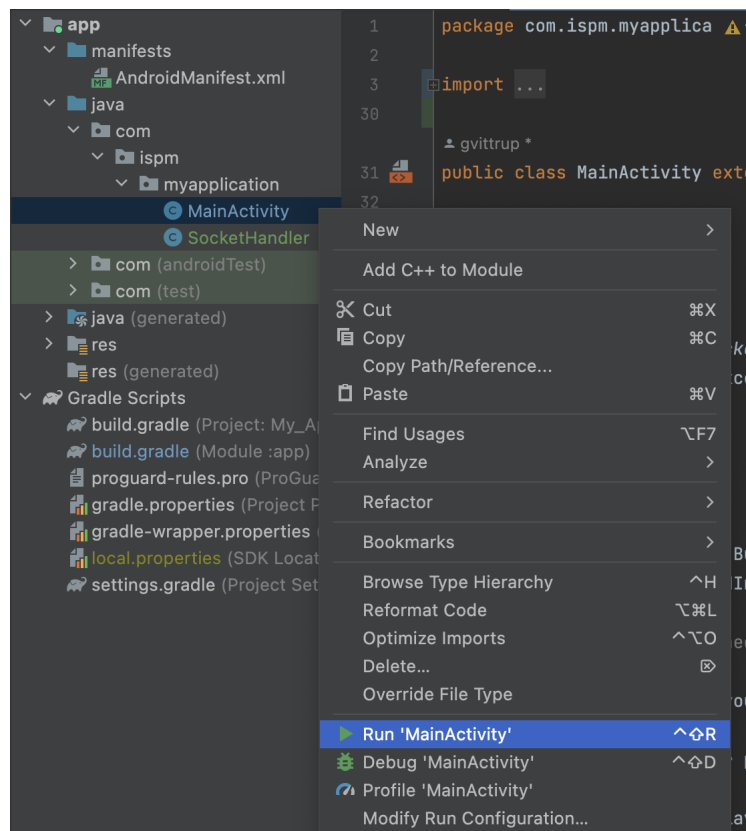


Figure 1: How to Compile MainActivity

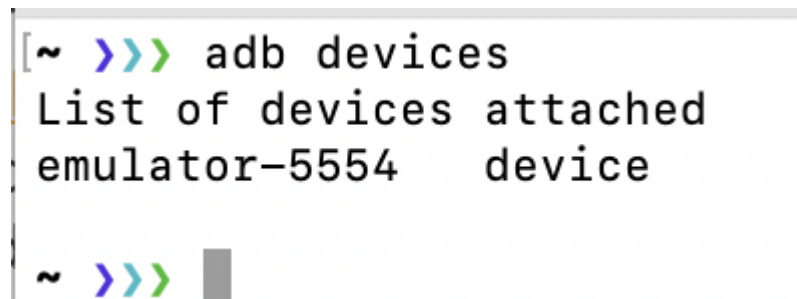
tab. Appium Server allows you to connect to an emulated device via your system.

Next, install the latest version of [Appium Inspector](#) on your respective device. Navigate to the link and locate the 'Releases' section on the right-hand side of the page. Below you will find a list of images beneath the 'Assets' tab. Appium inspector allows you to attach to your Appium Server session and inspect a given device and activity.

Next, install the latest version of [Node.js](#). Node.js is a runtime environment that allows us to host a server from our local machine. In this instance, we will use Node.js to host the remote server that our malicious activity connects to and sends information to. Navigate to the link and install the most

recent LTS version of Node.js. To validate that you have successfully installed Node.js, you can run the command 'node -v' in your terminal which should output the version that you installed.

Lastly, it is required that you install the [Android Debug Bridge](#) (adb) in order to interact with your device from the command-line. Navigate to the link and make sure you have installed the SDK Manager and platform-tools that will allow you to utilize this command. To test this, run your device via Android Studio and then run the command 'adb devices' to confirm that you see your device running on your machine.

A terminal window with a dark background and light-colored text. The prompt is a tilde followed by three colored chevrons (blue, green, red). The command 'adb devices' has been entered. The output shows 'List of devices attached' followed by 'emulator-5554' and 'device' on the next line. The prompt is shown again at the bottom of the terminal window.

```
[~ >>> adb devices
List of devices attached
emulator-5554    device

~ >>>
```

Figure 2: Validating ADB Devices

Task Set 1: Getting Device Information & Sending it to a Remote Server

Task 1.1: Getting Device Information

The first step to this attack is to retrieve the information of the devices that your malicious activity is installed on. When a user opens your malicious application for the first time, your application will asynchronously send certain device information back to your remote server. This information being sent back contains two main components: a list of installed packages on the users device and the device resolution. The installed packages allow you to determine a target application that you can surmise a user will use on their target device. From here, you will take the target application and replicate it remotely within Android studio. The importance

behind retrieving the resolution from the device is so you can match the native UI exactly. To do this successfully, you will use several built-in Android classes and functions. In order to get a list of packages, you can use the [PackageManager](#) class. The list can be retrieved using either the API function `getInstalledPackages` or the API function `getInstalledApplications`. The resolution can be achieved by utilizing the [DisplayMetrics](#) class. Retrieve this information utilizing the hints above and log this information to the Logcat console. When retrieving the device resolution, make sure that you are returning the *real* resolution.

Note: In order to log to the console, you will need to use Android's Log function (Logcat), rather than Java's `console.log`.

Action

1.1.1 - Log to the console a list of installed packages along with the device resolution and submit a screenshot of both your output and your code.

To go a step further, you can filter this list of packages to exclude any native system applications or services that would not be of interest to you as an attacker. This way, you would reduce the amount of network traffic being sent to and from the device, as well as reduce the amount of unnecessary information. This can be done in a multitude of ways.

Action

1.1.2 - What is one way you might be able to filter out some undesirable applications from the list of package names?

Task 1.2: Creating the Remote Server

After successfully retrieving the device information within Android Studio's console, the next step is to package it up and send the information to a remote server. For this, we will be using the java library `socket.io`. `Socket.io` allows us to use Node.js to host a server, create a socket, and connect to that endpoint from another program. Here, we will explore connecting

our emulated device to this server when our malicious application is running.

Follow the provided video part 1 on creating the Node.js server, titled "P1: Node.js Server."

Action

1.2.1 - Create an active Node.js server and connect to it via the localhost address in your browser. Submit a screenshot of the output of this page along with the output in your terminal logging what port your server is listening on.

Task 1.3: Connecting and Communicating w/ the Remote Server

Now that you have successfully hosted your server on your local machine, its time to use socket.io to create a socket (or an endpoint) on the server for us to connect to from within our Android device.

Follow the second provided video tutorial, titled "P2: Socket.io," on how to set up and validate that your device is connecting. When done correctly, your server log should read that it has a new device connected.

Action

1.3.1 - Create an active connection to the remote server and submit a screenshot of your server log logging the new socket connection.

After successfully connecting to the device, the next step is sending the information remotely. You may decide how you send this over. For the list of package names, you will want to send an array, and you can choose to either handle this on the server side or the client side.


```
PS C:\Users\gunna\AndroidStudioProjects\VenomAttack\backend> npm run start

> backend@1.0.0 start
> node server.js

Server is running
New socket connection: JH0XwH739olgHat2AAAB
New socket connection: ORrVUwH04EJC0DK3AAAD
|
```

Figure 3: Two Successful Socket Connections

Action

1.3.2 - Send over the device information retrieved from your application, log it out to the console, and submit a screenshot that shows your socket connection, the real device resolution, and the list of package names.

```
Server is running
New socket connection: zkS49kpR6xWEXoe8AAAB
2340 x 1080
com.google.android.youtube
com.ispm.venomattack
com.google.android.googlequicksearchbox
com.android.documentsui
com.google.android.apps.messaging
com.android.contacts
com.android.camera2
com.google.android.deskclock
com.google.android.gm
com.google.android.music
com.google.android.dialer
com.google.android.apps.docs
com.google.android.apps.maps
com.android.chrome
com.google.android.videos
org.chromium.webview_shell
com.google.android.apps.photos
com.google.android.calendar
com.android.settings
com.android.calculator2
com.google.android.apps.wallpaper
com.android.traceur
```

Figure 4: Server Log of Device Information

Task Set 2: Recreating App w/ Appium and Creating Patchfile

Task 2.1: Accessing Device via ADB Shell

By accessing the device via the ADB shell, you open the door to having much more control over the device you're interacting with, most importantly being the developer tools. While the utility of the adb is incredibly extensive, our application will be rather simple.

In this particular instance, we will use it in order to retrieve the launch activity of a specific page of a desired application. In this phase of the attack, the idea is that the attacker has already retrieved device information remotely from an unsuspecting user. Now it is up to the attacker to mimic the application in order to create a fake activity that would steal user information. In many cases, in order to do this, the attacker will simply utilize an emulated device on their own machine. While in our context the emulated device will be the same device, the idea behind the process still stands.

In Android studio, make sure your emulated device is actively running. Within your terminal, run the command 'adb devices'. You should see an output reading you the name of your emulator, as pictured in the setup section of this document. In order to run commands on this device, you will use the following command:

```
$ adb -s <name_of_device>
```

The -s flag allows us to specify the specific device you will be utilizing.

Running this command by itself will print out the version of the Android Debug Bridge you have along with the path where it is installed and a list of commands on how to use it. For example, to return a list of packages like we did in the first step, you can simply run the command:

```
$ adb -s <name_of_device> shell pm list packages
```

For the sake of this exercise, we will use the adb shell to retrieve the launch

activity and package of a specific screen within our target application selected from the list of applications retrieved in the first step.

For the target application itself, we recommend using the same target application used in the PHYjacking Lab, also by us. To use this application, download the project files for the PHYjacking Lab from [this Google Drive link](#). Once downloaded, open up the project files and locate the Android-Manifest.xml file under PHYJacking-lab/app/src/main/ and changing the TransferPayActivity's exported attribute from false to true and then running the java file named "MainActivity.java" under the file path PHYJacking-lab/app/src/main/java/com/ispn/app/. This will install the app to the running emulation device.

In order to get the appropriate launch details with adb, navigate to the target application on your emulated device. In most attacks, the desired portion of an application in order to successfully perform a phishing attack is the login page. Navigate to the login page of the target application. Now, navigate back to your terminal and run the command as one line:

```
$ adb shell "dumpsys window windows | grep -E
'mCurrentFocus|mFocusedApp'"
```

This command will output the current focused app and activity on the emulated device, with the first portion of the output being the package name and the second portion being the launch activity. This information will be paramount to connecting to the application via Appium.

Action

2.1.1 - Retrieve the launch activity of the target application and the respective login page that you will utilize in the Appium server and submit a screenshot of your terminal output.

```
~ >>> adb shell "dumpsys window windows | grep -E 'mCurrentFocus|mFocusedApp'"
mCurrentFocus=Window{42a12a3 u0 com.android.launcher3/com.android.launcher3.Launcher}
mFocusedApp=AppWindowToken{314b36 token=Token{dfe4d1 ActivityRecord{988c7f8 u0 com.android.launcher3/.Launcher t1094}}}
~ >>> adb shell "dumpsys window windows | grep -E 'mCurrentFocus|mFocusedApp'"
mCurrentFocus=Window{10a84bc u0 poc.phyjacking.transattack/poc.phyjacking.raceattack.MainActivity}
mFocusedApp=AppWindowToken{892b4b0 token=Token{d667ff3 ActivityRecord{df73262 u0 poc.phyjacking.transattack/poc.phyjacking.raceattack.MainActivity t1125}}}
```

Figure 5: ADB Device Information

Task 2.2: Connecting to Device Activity via Appium

From here, the next step requires us to take the information we garnered and input it into the Appium server. The information you've garnered in the previous exercises will be necessary to complete this next step. Load into the Appium Server GUI that you downloaded in the setup section of this document. You will be given two inputs: a host and a port. Set your host to "0.0.0.0" and your port number to an arbitrary and unused port. Then select "Start Server". You should be provided with a terminal window from the Application that reads to you the server is running.

Now open the Appium Inspector tool. You will see a left hand set of parameters and a right-side JSON representation. In total, you will want to have 6 parameters that you pass to the server. These parameters will all be of type text and will read as follows:

1. platformName: "android"
2. platformVersion: "9"
3. deviceName: "Pixel-5"
4. automationName: "UiAutomator2"
5. appPackage: ""
6. appActivity: ""

Where the last two parameters are provided by the information retrieved in the previous step. For the parameters at the top, set your Remote Host to "0.0.0.0", the Remote Port to the port number you provided to the server, and your Remote Path to "/wd/hub/".

Once you have all these entered, select "Start Session". If done so successfully, your Appium Server log should read to you the connection of your inspector with the parameters you passed, as noted in Figure 7 on page 14.

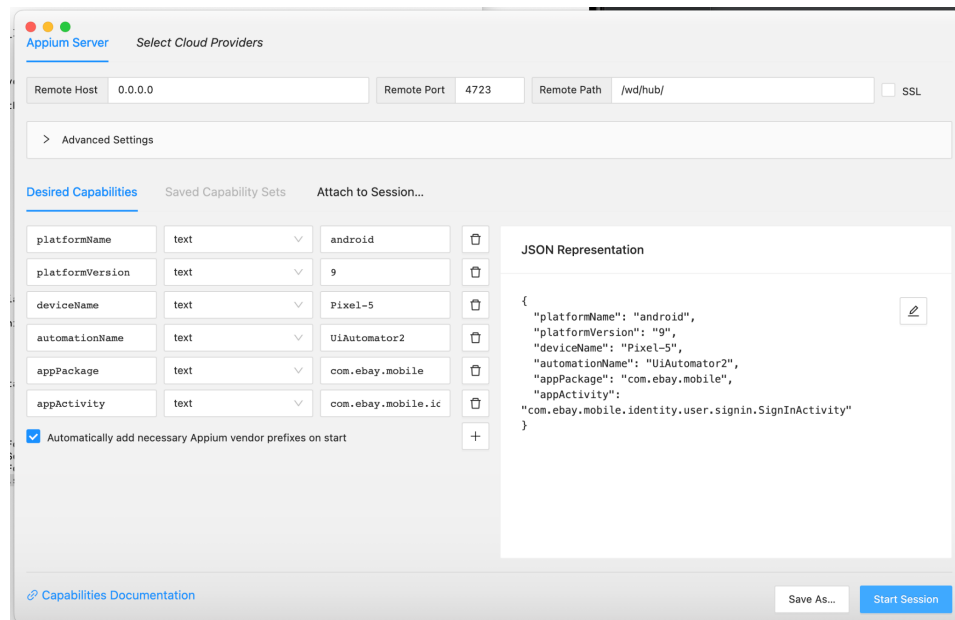
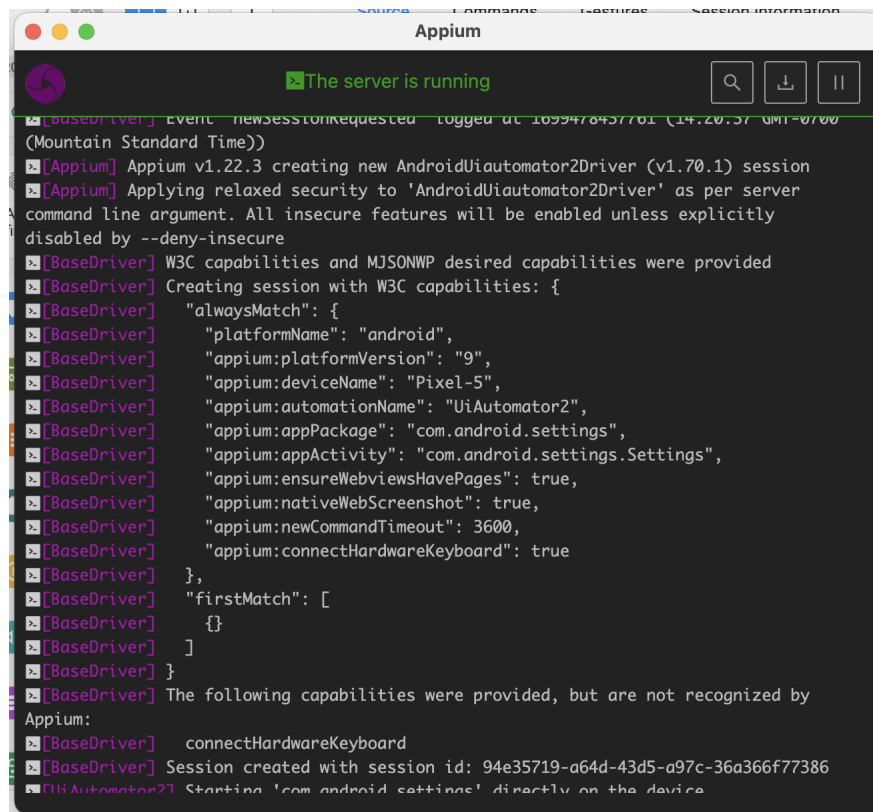


Figure 6: Appium Connection Example

Action

2.2.1 - Connect to the device with the correct parameters and submit a screenshot of your Appium server log reading to you the successful connection.

Once you have successfully connected, Appium will open the inspector and provide you a view of the target application and the page you are trying to copy. In the viewport on the left, you are able to interact with the application as if it were a normal emulator, toggling between different pages and buttons. This allows you to navigate throughout the application and explore the layouts of each different one. Ideally, you'll want to stay on the page you targeted and inspect the elements within the page. As you can see on the right side of the GUI, you will have an XML layout of the application that serves as a framework for you to recreate the UI of the target application. Furthermore, this is where you will want to take a screenshot of the target application to mimic the background. Some elements will need to be added, as required by Android Studio, such as `layoutWidth` and `layoutHeight`. Though you need to add certain elements



```
Appium
[The server is running]
[BaseDriver] Event: newSessionRequested logged at 1659476457761 (-14.20.37 GMT-0700 (Mountain Standard Time))
[Appium] Appium v1.22.3 creating new AndroidUiautomator2Driver (v1.70.1) session
[Appium] Applying relaxed security to 'AndroidUiautomator2Driver' as per server command line argument. All insecure features will be enabled unless explicitly disabled by --deny-insecure
[BaseDriver] W3C capabilities and MJSONWP desired capabilities were provided
[BaseDriver] Creating session with W3C capabilities: {
[BaseDriver]   "alwaysMatch": {
[BaseDriver]     "platformName": "android",
[BaseDriver]     "appium:platformVersion": "9",
[BaseDriver]     "appium:deviceName": "Pixel-5",
[BaseDriver]     "appium:automationName": "UiAutomator2",
[BaseDriver]     "appium:appPackage": "com.android.settings",
[BaseDriver]     "appium:appActivity": "com.android.settings.Settings",
[BaseDriver]     "appium:ensureWebviewsHavePages": true,
[BaseDriver]     "appium:nativeWebScreenshot": true,
[BaseDriver]     "appium:newCommandTimeout": 3600,
[BaseDriver]     "appium:connectHardwareKeyboard": true
[BaseDriver]   },
[BaseDriver]   "firstMatch": [
[BaseDriver]     {}
[BaseDriver]   ]
[BaseDriver] }
[BaseDriver] The following capabilities were provided, but are not recognized by Appium:
[BaseDriver]   connectHardwareKeyboard
[BaseDriver] Session created with session id: 94e35719-a64d-43d5-a97c-36a366f77386
[UiAutomator2] Starting 'com.android.settings' directly on the device
```

Figure 7: Successful Server Connection

to make this compatible within Android Studio, it provides a quick and easy method that gives every element within the screen. This will serve as your malicious application in order to trick an unsuspecting user.

Action

2.2.2 - Inspect the layout of your target application. What is the layout of your file? Submit a screenshot.

Note: If you receive an error where the inspector reads “Could not obtain screenshot: Unable to capture the screen”, navigate to your terminal and run the command:

“adb root”. This will provide you root access and remove the security flag

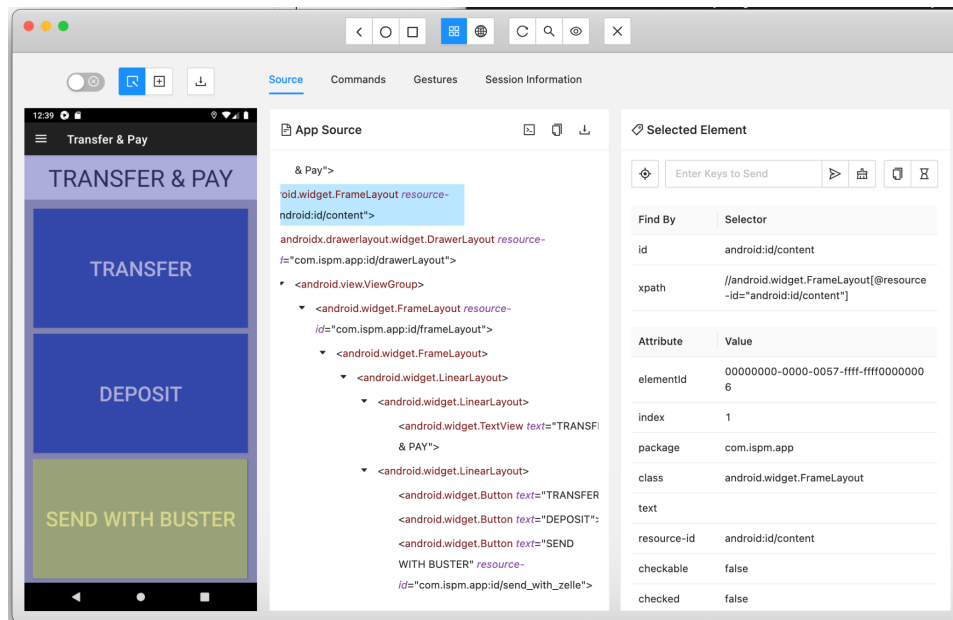


Figure 8: Inspecting Application

that causes this issue.

Task 2.3 Generating Fake UI & Patchfile

Many apps today use hot patch frameworks that allow patch files to be sent over a server and installed directly to the application. An example of this framework that was explored within the research paper was Tinker. In our exploration, we will be using the Android Studio's built in functionality that permits us the ability to create and apply them more easily. While we will not be sending this patch over a network and applying it remotely, it is important to understand the process to successfully mimic the attack.

Copy the XML file generated from the Appium Inspector and import it into the `AndroidLayout.xml` file. Here is where you will make your respective changes to make the XML compatible with Android Studio's format. After making a few additions and changes, your malicious activity should now very closely resemble the target application you chose to attack.

Action

2.3.1 - Take a screenshot of the target application on Appium. Use this as the target image for the layout in Android Studio. Submit a screenshot of both your malicious activity and of the target activity that was replicated.

Now that you have successfully mimic'd the target application, it is time to generate your patch file. On mobile devices, patch files do not re-install class files (such as java or kotlin). Instead, they reinstall the library and dependency files, such as your XML layout, your manifest files, or your build packs. This feature is what makes it possible for us to create a completely fake application that updates and adapts without the need for reinstallation or redistribution. Within Android Studio, stage your changes to the activity_main.xml file—but **DO NOT** commit your changes—in your view finder, navigate to Git -> Patch -> Create Patch from Local Changes and select the xml file that you staged.

Action

2.3.2 - Generate a patch file with your newly updated XML layout for your application and submit a screenshot of both the successful alert message along with the generated layout.

Task Set 3: Game Theory

Video: [The Purification Theorem](#)

In this task set, we are going to explore two game theory concepts: cutpoint strategies and the purification theorem. This will utilize some basic statistical knowledge such as probability and cumulative density functions.

In a game of cutpoint strategies, there is a continuum of possible types, where some action could cost anything between a lower bound and an upper bound. The information is incomplete, as each player's cost is private information. A cutpoint equilibrium helps us identify what every type of

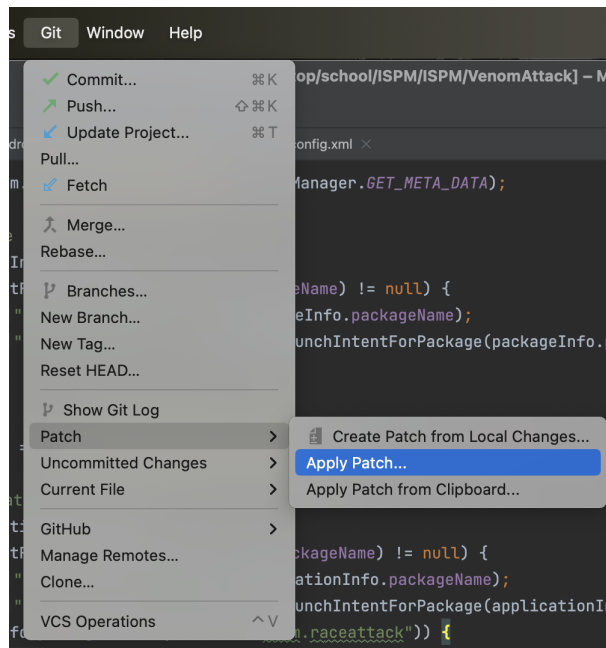


Figure 9: How to Generate a Patch

player will do. This cutpoint follows the rule that all types below a cutpoint take one action and all types above it take the opposite action.

Action

3.1.1 - If we were to consider the CDF independently drawn in Figure 10 from a symmetric cumulative distribution that is continuous and strictly increasing, what would the cutpoint (c^*) be equal to?

Now lets calculate the cutpoint equilibrium using the table descried in Figure 11. To start off, we need to calculate each players cutpoint equation.

Player 1's Cutpoint:

$$\varepsilon_{\Theta_1} > p(-1) + (1 - p)(2)$$

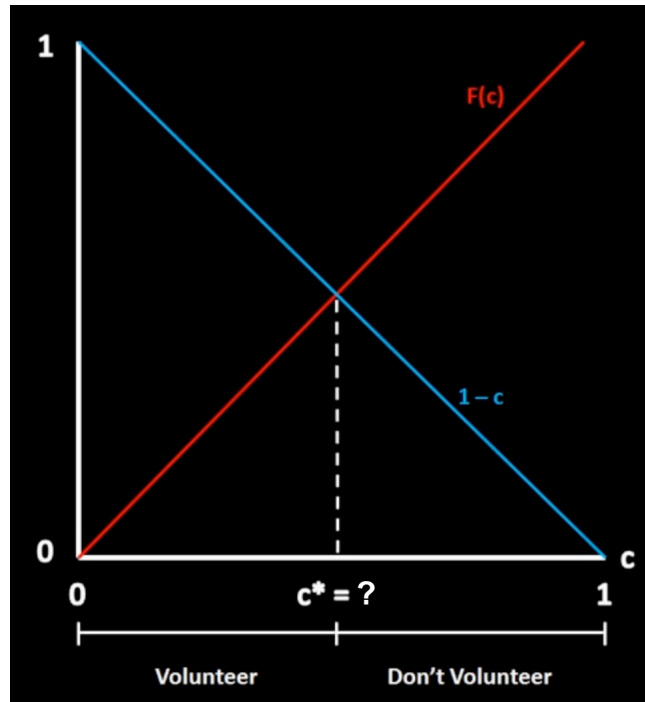


Figure 10: Continuous Cutpoint CDF

$$\Theta_1 > \frac{(2 - 3p)}{\varepsilon}$$

The system of equations above represents the probability equation that player 1 will play up given the probability, p , that player 2 plays left. Now let's calculate player 2's cutpoint.

Player 2's Cutpoint:

$$\varepsilon\Theta_2 > q(-1) + (1 - q)(4)$$

$$\Theta_2 > \frac{(4 - 5q)}{\varepsilon}$$

This will be the probability equation that player 2 will play left given player 1's probability, q , that player 1 plays up. Now let's recall from basic probability theory that if Θ is distributed uniformly from -1 to 1, then the probability that drawing a value smaller than x is:

$\Theta \sim U[-1,1]$
 $\varepsilon > 0$

		Player 2	
		Left	Right
Player 1	Up	$\varepsilon\Theta_1, \varepsilon\Theta_2$	$\varepsilon\Theta_1, -1$
	Down	$-1, \varepsilon\Theta_2$	$2, 4$

Figure 11: Game Theory Table

$$\frac{x+1}{2}$$

Therefore the probability of drawing a value of Θ_1 greater than x is:

$$1 - \frac{x+1}{2}$$

This is just essentially writing out the cumulative distribution function. If you were to plug -1 into the first equation, you would expect a probability of 0, as -1 is the lower bound of the distribution. If you were to plug in 1, it is the upper bound of the distribution and you are guaranteed to draw something less than it, thus your output would be 1.

In order for us to calculate the cutpoint equilibrium, we need to find our probabilities, p and q . In order to do this, we first must calculate our system of equations for p and q by substituting the inequality derived in the previous parts for x in the cumulative distribution function.

Action

3.1.2 - Substitute each inequality in for x and derive the system of equations for both p and q . What are these equations?

Hint: Set one of the variables equal to the cumulative distribution function and substitute the opposing variable in for x .

Now that you have your system of equations, we can solve for the probabilities p and q . Substitute the equation for one probability into the respective probability of the other equation and vice-versa. You should be left with two final equations, with the only unknown variable being ε . If you recall, ε is a number that is greater than but converging to 0.

Action

3.1.3 - Substitute 0 in for ε to calculate the probabilities for both p and q . What are the probabilities of p and q ?

These two values are your cutpoint equilibria. As this game converges from an incomplete information game to a complete information game, then the equilibrium probabilities of the players taking particular actions exactly match the probabilities from the complete information games mixed strategy nash equilibrium.

Submission

Submit a lab report utilizing the provided template to your instructor, which should include screenshots, thoughts, challenges, and lessons learned from the lab. Furthermore, zip and attach your project files with the relevant java files for submission.

Summary

In this lab we've discovered how to gather information from an Android app and send it to a remote server, in our case a NodeJS Express server, utilize Appium to automatically generate UI layouts, and how to create a patch file to update an application without needing to reinstall it. All together, these methods allow attackers to be able to update attack payloads without the need for re-installation or redistribution, further increasing the impact of activity hijack attacks.

Trouble Shooting

Refactoring the Project Name

- Make sure you change the respective lines within build.gradle and AndroidManifest.xml to match what you refactored your project to. Then, select File -> Invalidate Caches -> Restart, and your project should rebuild with the correct dependencies. If this does not work, you can select Build -> Clean Project and Build -> Rebuild Project and then retry invalidating the caches. This should fully correct your problem.

References

- [1] Lingling FAN Pengfei GAO Fu SONG Min YANG Pu SUN, Sen CHEN. Venomattack: automated and adaptive activity hijacking in android. *Frontiers of Computer Science*, 17(1):171801, 2023.